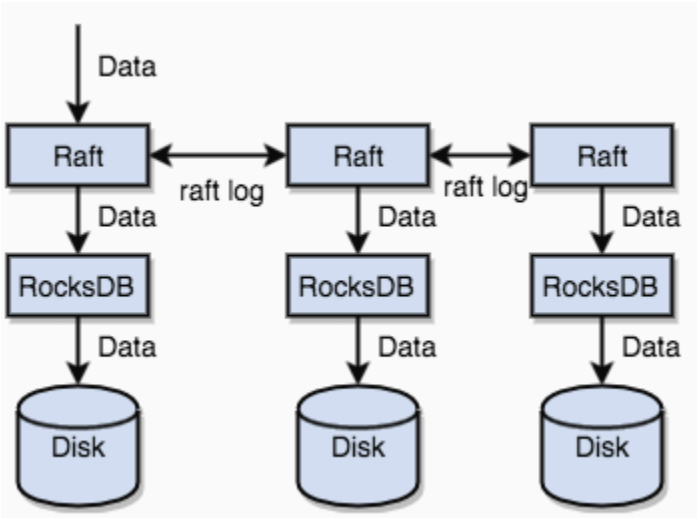
Midterm Project Part 04: TiDB (Titanium Database)

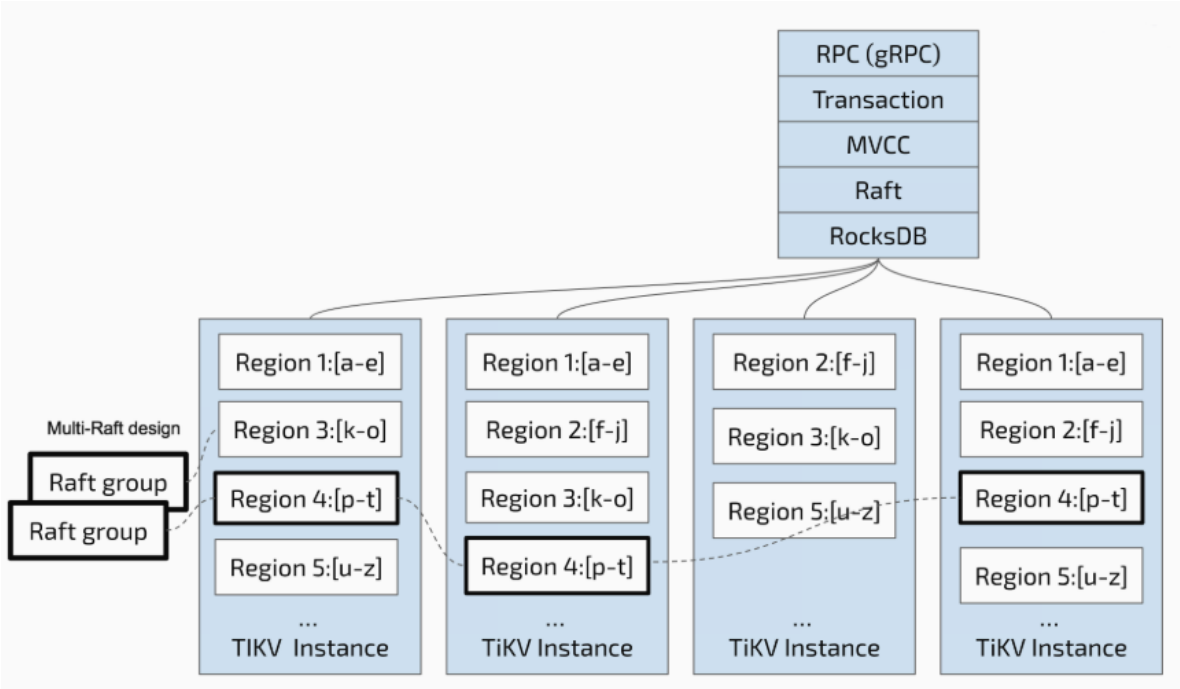
4.1 1. Architecture Overview

How does the system distribute and replicate data across nodes?

- What is the storage engine (LSM tree, B-tree, custom)?
  TiDB's storage engine (TiKV) is based on RocksDB, which uses an LSM-tree (Log-Structured Merge Tree) design.

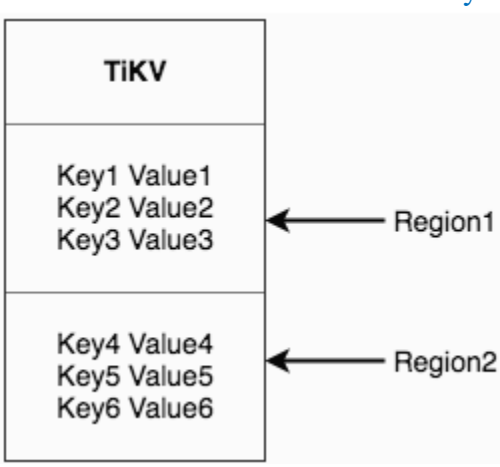
- What consensus protocol does it use (Raft, Paxos, Calvin, custom)?
  TiKV uses the Raft consensus protocol for data replication. Each write is recorded as a Raft log entry and replicated to multiple nodes in a Raft group. A write is considered successful once a majority (quorum) of nodes have replicated the log, ensuring strong consistency and fault tolerance.



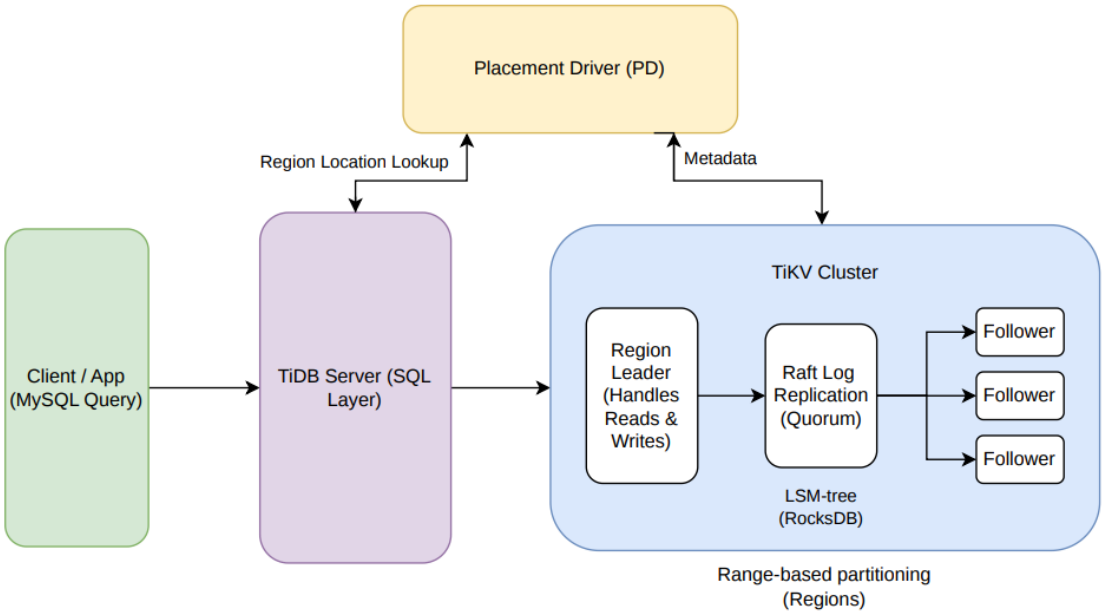


- What are the node roles (leader/follower, coordinator/worker, homogeneous)?
  TiDB uses a leader–follower model within TiKV (via Raft), where each region has a leader and followers. The overall system is heterogeneous, with separate roles for TiDB (compute), TiKV (storage), and PD (management).

- How is data partitioned (range-based, hash-based, reference tables)?
  TiDB (via TiKV) partitions data using range-based partitioning, where the keyspace is split into contiguous ranges called Regions. These Regions are automatically split and distributed across nodes for scalability.



Required: Include at least one original architecture diagram showing how data flows through the system.



4.2 2. CAP Theorem Positioning

Where does the system sit on the CAP spectrum?

- Is it CP (consistent + partition-tolerant) like PostgreSQL, or AP (available + partition-tolerant) like Cassandra?
- What happens during a network partition—does the system favor consistency or availability?
  TiDB (via TiKV) follows the **CP model** because it uses the Raft consensus protocol, which requires a majority (quorum) of nodes to agree before a write is committed. This ensures **strong consistency**, but if a network partition occurs and a quorum cannot be reached, the **system may sacrifice availability**.
- Can the user configure this trade-off, or is it fixed by design?
  TiDB is fundamentally a CP system due to its use of Raft consensus, so strong consistency is enforced by design. However, users can make limited trade-offs by adjusting settings such as replica placement or using follower reads (stale reads) to improve availability and performance for certain workloads. These options do not change the core CP nature of the system.
- How does this compare to PostgreSQL (Parts 1–2) and Cassandra (Part 3)?

PostgreSQL offers strong consistency but limited scalability, Cassandra offers high availability with eventual consistency, and TiDB provides strong consistency with horizontal scalability, combining benefits of both while remaining CP.

4.3 3. SQL Compatibility

How complete is the SQL support?

- Which SQL standard does it target (PostgreSQL, MySQL, ANSI SQL)?
  TiDB targets MySQL compatibility.
- What SQL features are missing or limited (stored procedures, triggers, CTEs, window functions)?
  TiDB does not support stored procedures or triggers, and some features are limited, such as certain query optimizations and execution plans. It supports CTEs and window functions, but with some performance or optimization limitations in distributed queries.
- Can existing PostgreSQL or MySQL applications migrate with minimal changes?Existing MySQL applications can migrate to TiDB with minimal changes, since TiDB is MySQL-compatible. However, PostgreSQL applications typically require more modifications because TiDB does not target PostgreSQL syntax or protocol.
- Are there any query patterns that perform poorly in the distributed setting?
  Yes. In TiDB, query patterns that require heavy cross-node data movement—such as large joins, global sorting, and some COUNT(DISTINCT) queries—can perform poorly in a distributed setting. Some complex CTE and window-function-related queries can also be less optimized.

4.4 4. Scaling Model

How does the system scale horizontally?

- Is sharding automatic or manual?
  Sharding is automatic in TiDB. Data is split into key ranges called Regions, which are distributed across TiKV nodes. The system automatically handles splitting, balancing, and movement of data without manual intervention.
- How does the system handle rebalancing when nodes are added or removed?
  When nodes are added or removed, the Placement Diver detects changes in cluster load and automatically redistributes Regions across TiKV nodes. It moves Region replicas and transfers leadership to balance storage and workload evenly, ensuring high availability and fault tolerance.
- What is the unit of distribution (tablet, shard, chunk, vnode)?

The unit of distribution in TiDB is called a Region, which is a range of keys. Regions are distributed across TiKV nodes and replicated using Raft.

- How does it compare to the manual sharding you did in Part 2?
  TiDB uses automatic sharding, while Part 2 (PostgreSQL) used manual sharding.

4.5 5. Consistency & Transactions

What consistency and transaction guarantees does the system provide?

- What isolation levels are supported (read committed, snapshot, serializable)?
  TiDB supports Snapshot Isolation (SI), which is exposed as REPEATABLE READ (default), and also supports Read Committed. It does not support Read Uncommitted or full Serializable isolation.
- How are distributed transactions handled (two-phase commit, Paxos, Raft, Calvin)?
  TiDB uses a Two-Phase Commit (2PC) protocol to coordinate distributed transactions across multiple nodes, while Raft is used underneath for data replication and consistency within each Region.
- What is the performance cost of strong consistency (latency, throughput)?
  Strong consistency increases latency and reduces throughput because multiple nodes must agree before completing a transaction.
- How does this compare to PostgreSQL's serializable isolation and Cassandra's tunable consistency?
  TiDB provides strong consistency by default, similar to PostgreSQL, but with more overhead because it is distributed. PostgreSQL's serializable isolation also ensures strong consistency but runs on a single node, so it is usually faster. Cassandra, on the other hand, allows tunable consistency, meaning users can choose between faster performance or stronger consistency depending on their needs.

4.6 6. TechRetail Fit

Would this system be a good replacement for any component in the Parts 1–3 architecture?

- Which component could it replace (sales shards, Cassandra ingestion, warehouse, or multiple components)?
  Instead of running multiple PostgreSQL shard containers and manually splitting the data, the system could use a TiDB cluster where data is automatically distributed across TiKV nodes. In docker-compose, this would mean removing the separate sales shard services and replacing them with TiDB components like a TiDB server, PD, and TiKV nodes. TiDB could also reduce the need for a separate warehouse, since it can support both OLAP and OLTP in the same system.
- What would improve compared to the current architecture?

The biggest improvement would be simplifying the sales layer. Instead of manually splitting data and routing queries across multiple databases, TiDB would automatically distribute and rebalance the data. This makes reads and writes easier to manage since the application connects to a single system instead of multiple shards. It could also reduce ETL complexity if some reporting queries are run directly on TiDB instead of moving data to a separate warehouse.

- What would get worse or become more complex?
  TiDB would introduce more distributed-system complexity, such as managing Raft, distributed transactions, and cluster coordination. Strong consistency can also increase latency compared to simpler or more available systems like Cassandra. In addition, if the current warehouse is highly optimized for analytics, replacing it with TiDB may not always give the same performance for heavy analytical workloads.
- Would the ETL pipeline need to change?
  Yes. If TiDB replaced the sales shards, the ETL pipeline would no longer need to extract data from each shard separately. Instead, it would connect to TiDB as a single data source, simplifying the extraction process and removing manual shard routing. If TiDB also replaced the warehouse, part of the ETL pipeline could be reduced, since less data movement and fewer transformations would be needed.

Required: Include a comparison table showing your system vs PostgreSQL and/or Cassandra for the relevant use case.

Aspect	PostgreSQL (Sales Shards)	Cassandra (Ingestion)	TiDB
Architecture	Single-node / manually sharded	Distributed	Distributed
Sharding	Manual	Automatic (hash-based)	Automatic (range-based Regions)
Scalability	Limited	High horizontal scalability	High horizontal scalability
CAP Positioning	CP	AP	CP
Consistency Model	Strong (ACID)	Tunable (eventual consistency)	Strong (Raft-based)
Best for	Structured transactions, joins, normalized data	High write throughput, distributed data, scalability	Distributed transactional + hybrid workloads
Limitation	Hard to scale	Weaker consistency	Higher latency due to coordination
SQL Support	Full SQL	Limited (CQL)	MySQL-compatible SQL

References

PingCAP. TiDB Storage Documentation. https://docs.pingcap.com/tidb/stable/tidb-storage/

PingCAP. Transaction Isolation Levels. https://docs.pingcap.com/tidb/stable/transaction-isolation-levels/

DBDB.io. TiDB Overview. https://dbdb.io/db/tidb